

23CSE455 Design Patterns

Case Study Report

Team: B13

Case study: Energy Management Application using FastAPI — Design Pattern Identification and Evolution in a FastAPI-Based Energy Management Application

Application: FastAPI Energy Management System for simulated load scheduling, optimization, renewable generation, tariffs, alerts, and runtime monitoring

Date: 2026-10-07

Submission repository: [23CSE455_DP_CaseStudy](#), team folder B13/

Students

Roll number	Student name	Batch
AM.SC.U4CSE23306	Anand Narayan	S7 CSE D
AM.SC.U4CSE23346	S H Saimadhav	S7 CSE D
AM.SC.U4CSE23365	Vaishnav P Nair	S7 CSE D
AM.SC.U4CSE23369	Vishnu M	S7 CSE D

1 Application and architecture

Original source: The baseline was supplied as `energy-management-system-main.zip`. No upstream repository URL, version tag, Git history, or license file was included. The archive SHA-256 is `A53D974449C1CCC0BA322267DB37BD7CDD626CC8E80C55C604EBEBF757FCC060`; it was inspected on 2026-10-07. The evolved source archive is `energy-management-system-part2.zip`, SHA-256 `19A9AAF2447540E38AB6E1667B664E64B535E36B2E8B23580181DF1A1725FD8C`.

Environment: Windows; Python 3.12.14; FastAPI 0.142.2; Pydantic 2.13.5; pytest 9.1.1; PuLP 3.3.2; httpx 0.28.1. The application uses an in-memory repository and simulated forecast data; no external database is required.

Scope: Inspected `ems/domain/models`, `ems/domain/ports`, `ems/application/services`, `ems/application/solvers`, `ems/infrastructure`, `ems/interface`, and `test`. Generated files, virtual environments, dependency packages, and caches were excluded. Part 1 evaluates the supplied baseline; Part 2 evaluates the evolved application; Part 3 evaluates isolated equivalent pattern problems.

Architecture: The application follows a layered Ports-and-Adapters structure. FastAPI REST routes and WebSocket handling form the interface layer. Application services coordinate portfolio changes, metric advancement, planning, and configuration. Domain models and ports describe portfolios, loads, metrics, forecasts, repositories, and energy configuration. Infrastructure adapters provide simulated forecasts and in-memory persistence. A normal request enters through FastAPI, resolves the shared runtime through dependency injection, calls an application service, updates domain state, and returns or broadcasts the serialized result.

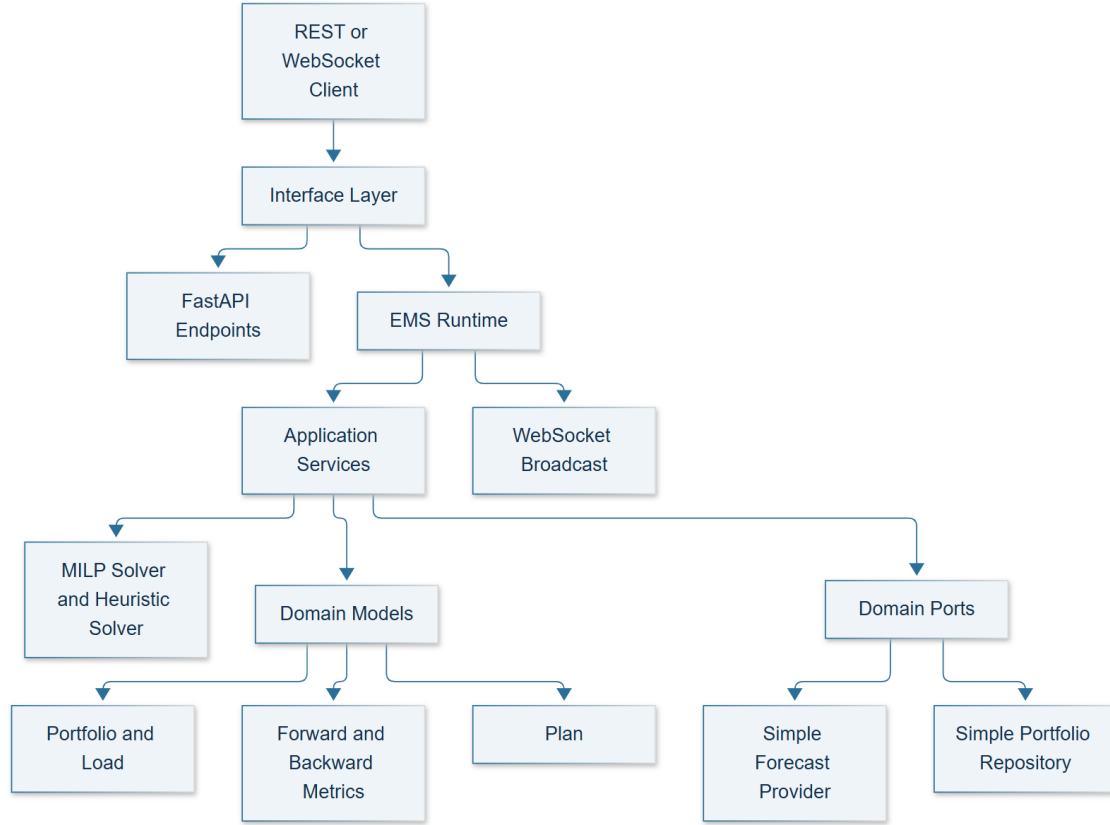


Figure 1: Baseline application architecture.

Run instructions: After placing the evolved source in `Part2/code`, create and activate a virtual environment, run `pipinstall-rrequirements-dev.txt`, execute `python-mpytest-qtest`, execute `python-mtest.test_milp_solver`, and start the server with `python-mems`. The verified automated result was 13 tests passed in 0.53 seconds; the MILP build-and-solve smoke test also completed. Swagger UI is available at <http://localhost:8000/docs> while the server is running.

2 Part 1 Pattern discovery

Method: Candidates were located by tracing abstractions, construction points, injected dependencies, interchangeable algorithms, subscriber collections, and calls across the interface, application, domain, and infrastructure layers. Every confirmed collaboration was checked for intent, participants, delegation, runtime behavior, and exact source locations. Capability names alone were not treated as proof of a design pattern. Results are recorded in `data/patterns.csv`, `data/pattern_counts.csv`, and `Part1/evidence/patterns/pattern_mapping.md`.

2.1 Totals at the baseline

Confirmed pattern types: 5 distinct types: Ports and Adapters, Repository, Dependency Injection, Strategy, and Observer.

Confirmed pattern instances: 6 collaborations: two Ports-and-Adapters boundaries and one instance each of Repository, Dependency Injection, Strategy, and Observer.

Concrete participant classes: 3 distinct application-defined concrete classes in variable runtime roles: `SimpleForecastProvider`, `SimplePortfolioRepository`, and `EMSRuntime`. Solver

strategies are functions, and connected WebSocket clients are runtime framework objects rather than additional application classes.

Unique participant symbols: 13 explicitly named application symbols after shared participants are counted once: `ForecastProvider`, `SimpleForecastProvider`, `ems_service.advance`, `PortfolioRepository`, `SimplePortfolioRepository`, `manage_portfolio.create_portfolio`, `get_runtime`, `EMSRuntime`, `websocket_endpoint`, `deploy_plan.generate_plan`, both solver solve functions, and `EMSRuntime.broadcast`. REST route handlers are treated as one dependent group rather than inflated into separate pattern instances.

Rejected and uncertain candidates: No separate baseline Factory, alert Observer, smart-meter Adapter, or reporting pattern was confirmed. Dataclass `default_factory` usage was rejected as language-library construction syntax rather than a Factory pattern. WebSocket broadcasting was confirmed as Observer-style runtime monitoring, but it was rejected as evidence of a consumption-alert system because no threshold rule or alert record existed. Observation scope is limited to the supplied application source.

2.2 Pattern inventory

Pattern type	Instance names	Instances	Concrete classes
Ports and Adapters	Forecast Provider Boundary; Portfolio Persistence Boundary	2	2
Repository	Portfolio Repository	1	1
Dependency Injection	FastAPI Runtime Injection	1	1
Strategy	Solver Selection	1	0 (2 functions)
Observer	WebSocket Runtime Updates	1	1 plus runtime clients
Total		6	3 distinct application classes

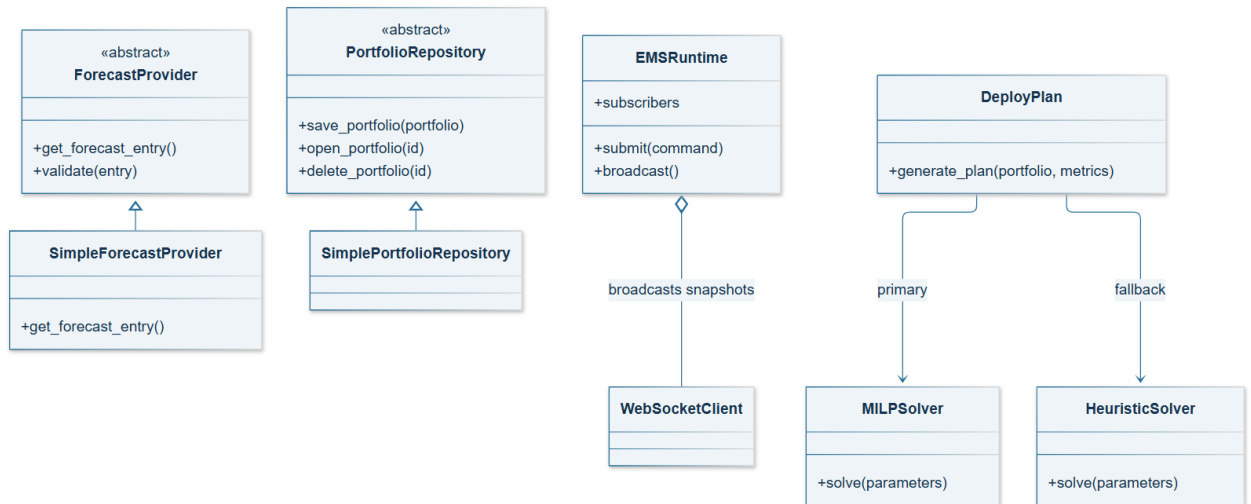


Figure 2: Baseline pattern participant UML.

2.3 One record for each pattern instance

2.3.1 Forecast Provider Boundary

Pattern type and category: Ports and Adapters; architectural pattern.

Purpose and origin: Keeps forecast generation behind a domain-facing port. It originates in the supplied application.

Participant mapping

Pattern role	Actual class, interface and function	File and line range
Port	ForecastProvider	Part1/code/ems/domain/ports/forecast_provider.py:5-20
Adapter	SimpleForecastProvider	Part1/code/ems/infrastructure/simple_forecast_provider.py:6-18
Client	ems_service.advance	Part1/code/ems/application/services/ems_service.py:46-50

Evidence: Application services depend on the forecast abstraction while the infrastructure adapter implements simulated forecast generation.

Diagram: Editable source: Part1/evidence/patterns/pattern_uml.mmd; rendered image: Part1/evidence/patterns/pattern_uml.png.

Verification: Confirmed by mapping the port declaration, implementation, and client call. Reviewer: Team B13. Alternative rejected: direct service-owned forecast generation.

2.3.2 Portfolio Persistence Boundary

Pattern type and category: Ports and Adapters; architectural pattern.

Purpose and origin: Separates application services from the in-memory persistence mechanism. It originates in the supplied application.

Participant mapping

Pattern role	Actual class, interface and function	File and line range
Port	PortfolioRepository	Part1/code/ems/domain/ports/portfolio_repository.py:8-21
Adapter	SimplePortfolioRepository	Part1/code/ems/infrastructure/simple_portfolio_repository.py:7-17
Client	manage_portfolio.create_portfolio	Part1/code/ems/application/services/manage_portfolio.py:9-17

Evidence: The service calls a domain-facing repository contract; the concrete adapter owns the storage collection and persistence operations.

Diagram: The collaboration is included in Part1/evidence/patterns/pattern_uml.mmd.

Verification: Confirmed by tracing create, get, and delete calls from the service to the port and concrete adapter. Reviewer: Team B13.

2.3.3 Portfolio Repository

Pattern type and category: Repository; architectural/domain pattern.

Purpose and origin: Centralizes portfolio persistence operations and prevents application services from manipulating the storage collection directly. It originates in the supplied application.

Participant mapping

Pattern role	Actual class, interface and function	File and line range
Repository contract	PortfolioRepository	Part1/code/ems/domain/ports/portfolio_repository.py:8-21
Concrete repository	SimplePortfolioRepository	Part1/code/ems/infrastructure/simple_portfolio_repository.py:7-17
Client	Portfolio-management service functions	Part1/code/ems/application/services/manage_portfolio.py:9-17

Evidence: Add, retrieve, and delete behavior is exposed through one repository contract. This instance shares participants with the persistence port-and-adapter boundary but expresses a separate repository intent.

Diagram: The collaboration is included in Part1/evidence/patterns/pattern_uuml.mmd.

Verification: Confirmed. Reviewer: Team B13. The shared participants are counted once in the unique-symbol total.

2.3.4 FastAPI Runtime Injection

Pattern type and category: Dependency Injection; framework idiom and architectural pattern.

Purpose and origin: Supplies one shared runtime to REST and WebSocket handlers without constructing it inside each handler. It originates in the supplied FastAPI application.

Participant mapping

Pattern role	Actual class, interface and function	File and line range
Provider	get_runtime	Part1/code/ems/interface/runtime.py:136-138
Injected object	EMSRuntime	Part1/code/ems/interface/runtime.py:11-138
Dependents	REST route handlers	Part1/code/ems/interface/endpoints.py:46-113
Dependent	websocket_endpoint	Part1/code/ems/interface/websocket.py:7-22

Evidence: FastAPI Depends(get_runtime) resolves the runtime and passes it to handler parameters.

Diagram: The dependency direction is shown in Part1/evidence/architecture/architecture.mmd.

Verification: Confirmed through route signatures and the provider function. Reviewer: Team B13.

2.3.5 Solver Selection

Pattern type and category: Strategy; GoF behavioral pattern expressed with functions.

Purpose and origin: Attempts MILP planning first and invokes a compatible heuristic fallback

when the primary result is unavailable. It originates in the supplied application.

Participant mapping

Pattern role	Actual class, interface and function	File and line range
Context	<code>deploy_plan.generate_plan</code>	Part1/code/ems/application/services/deploy_plan.py:16-34
Concrete strategy	<code>milp_solver.solve</code>	Part1/code/ems/application/solvers/milp_solver.py:124-135
Concrete strategy	<code>heuristic_solver.solve</code>	Part1/code/ems/application/solvers/heuristic_solver.py:205-308
Strategy interface	Not separately declared	Compatible function-level input/output contract

Evidence: The context delegates to the primary solver and conditionally delegates to the fallback through a compatible result contract.

Diagram: The functional Strategy mapping is shown in Part1/evidence/patterns/pattern_um1.mmd.

Verification: Confirmed as a strategy-like collaboration without inventing a missing interface class. Reviewer: Team B13.

2.3.6 WebSocket Runtime Updates

Pattern type and category: Observer; GoF behavioral pattern expressed through runtime client objects.

Purpose and origin: Maintains connected WebSocket subscribers and broadcasts application-state snapshots. It originates in the supplied application.

Participant mapping

Pattern role	Actual class, interface and function	File and line range
Subject / concrete subject	<code>EMSRuntime</code>	Part1/code/ems/interface/runtime.py:11-103
Observer interface	Not separately declared	WebSocket send contract
Concrete observers	Connected WebSocket clients	Runtime objects stored by <code>EMSRuntime</code>
Attach / detach	<code>websocket_endpoint</code>	Part1/code/ems/interface/websocket.py:7-22
Notify	<code>EMSRuntime.broadcast</code>	Part1/code/ems/interface/runtime.py:11-103

Evidence: The runtime owns a subscriber list and sends each snapshot to registered WebSocket clients.

Diagram: The Observer collaboration is shown in Part1/evidence/patterns/pattern_um1.mmd.

Verification: Confirmed as runtime state publication. It is not classified as a consumption-alert system because the baseline has no alert rule. Reviewer: Team B13.

3 Part 2 Evolution and refactoring

3.1 CH01 — Dynamic tariffs

Change and acceptance criteria: Introduce interchangeable flat, time-of-use, and dynamic tariff policies. The selected policy must populate `GRID_PRICE`; `PUT/tariff` must accept valid configurations and reject incomplete or unsupported configurations.

Before and after: Before: price values were generated directly by the forecast provider. After: `TariffStrategy`, `FlatTariff`, `TimeOfUseTariff`, and `DynamicTariff` calculate prices. Exact Git commits are unavailable because the supplied archive had no history. Evidence: `Part2/evidence/changes/Change1/implementation.diff`.

Implementation and tests: The provider delegates price calculation to its configured strategy while the solver input remains `GRID_PRICE`. Tests cover flat, peak-window, repeating-series, provider-integration, and invalid-request behavior. Command: `python-mpytest-qtest`; full-suite result: 13 passed in 0.53 seconds. Log: `Part2/evidence/changes/Change1/test_result.txt`.

Pattern effects: New instance: Dynamic Tariff Selection, Strategy. The existing Forecast Provider Boundary is modified; FastAPI Runtime Injection is extended to the new route; Solver Selection and the remaining baseline collaborations are preserved. Pattern instances increase from 6 to 7.

Refactoring: Smell: price-generation policy was embedded in the forecast adapter. Technique: Extract Strategy and dependency configuration. Behavior preservation: the provider still emits `GRID_PRICE`, so both solvers retain the same parameter contract.

Before/after metrics: Pattern types remain 5; instances increase from 6 to 7. Full-suite result after the completed evolution is 13/13. A separate comparable source-LOC measurement was not captured.

The figure displays three Swagger UI panels, each representing a different tariff configuration. Each panel includes a 'Curl' section with a `curl` command, a 'Request URL' field, a 'Server response' table, and a 'Response body' section.

- Left Panel (Flat Tariff):** The `curl` command sets `"kind": "flat"` and `"price": 0.15`. The response body is `{ "kind": "flat" }`.
- Middle Panel (Time of Use Tariff):** The `curl` command sets `"kind": "time_of_use"` with peak and off-peak prices. The response body is `{ "kind": "time_of_use" }`.
- Right Panel (Dynamic Tariff):** The `curl` command sets `"kind": "dynamic"` with a list of prices. The response body is `{ "kind": "dynamic" }`.

Figure 3: Swagger UI evidence for the three dynamic-tariff configurations.

3.2 CH02 — Renewable-energy integration

Change and acceptance criteria: Introduce named solar and wind sources with source-specific production rules, aggregate their values into `REN_PROD`, and expose configuration and summaries through `POST/energy-sources`, `GET/energy-sources`, and WebSocket state.

```
PS D:\B13\Part2\code\energy-management-system-updated> py -m ems
INFO:      Finished server process [15908]
INFO:      Started server process [17720]
INFO:      Waiting for application startup.
INFO:      Application startup complete.
INFO:      127.0.0.1:65158 - "GET /docs HTTP/1.1" 200 OK
INFO:      127.0.0.1:65158 - "GET /openapi.json HTTP/1.1" 200 OK
INFO:      127.0.0.1:58376 - "PUT /tariff HTTP/1.1" 200 OK
INFO:      127.0.0.1:50211 - "PUT /tariff HTTP/1.1" 200 OK
INFO:      127.0.0.1:50214 - "PUT /tariff HTTP/1.1" 200 OK
□
```

Figure 4: Terminal verification for dynamic-tariff behavior.

Before and after: Before: one aggregate simulated renewable value. After: `RenewableSource`, `SolarSource`, `WindSource`, and `EnergySourceFactory`. Exact commits are unavailable; evidence: `Part2/evidence/changes/Change2/implementation.diff`.

Implementation and tests: Solar uses a repeatable daylight profile; wind multiplies capacity by an availability series; the provider aggregates the sources. Tests cover both algorithms, factory creation, aggregation, REST output, and WebSocket output. Log: `Part2/evidence/changes/Change2/test_result.txt`.

Pattern effects: New Strategy instance: Renewable Production. New Factory instance: Renewable Source Creation. The Forecast Provider Boundary is modified; runtime injection and WebSocket output are extended; prior instances are preserved. Pattern types increase from 5 to 6 and instances from 7 to 9.

Refactoring: Smell: renewable behavior was represented by one undifferentiated value. Techniques: Extract Strategy and Introduce Factory. Behavior preservation: the solvers still receive the aggregate `REN_PROD` metric.

Before/after metrics: Pattern types: 5 to 6. Pattern instances: 7 to 9. Full-suite result after the completed evolution: 13/13. A separate comparable source-LOC measurement was not captured.

3.3 CH03 — Consumption alerts

Change and acceptance criteria: Publish completed per-load consumption events, record values strictly greater than a configurable threshold, expose `PUT/alerts/threshold` and `GET/alerts`, and include alerts in WebSocket snapshots.

Before and after: Before: completed grid and renewable draw were stored only as metrics. After: `ConsumptionEvent`, `ConsumptionSubject`, `ConsumptionObserver`, and `ConsumptionAlertObserver`. Exact commits are unavailable; evidence: `Part2/evidence/changes/Change3/implementation.diff`.

Implementation and tests: After backward metrics are updated, the application publishes a per-load event. The subject notifies subscribed observers; the alert observer stores records when combined draw is strictly greater than the threshold. Tests cover below, equal, and above-threshold cases, subscribe/unsubscribe, service integration, REST, and WebSocket behavior. Log: `Part2/evidence/changes/Change3/test_result.txt`.

Pattern effects: New instance: Consumption Threshold Alerts, Observer. Runtime Injection and WebSocket Runtime Updates are extended; all other existing instances are preserved. Pattern types remain 6 and instances increase from 9 to 10.

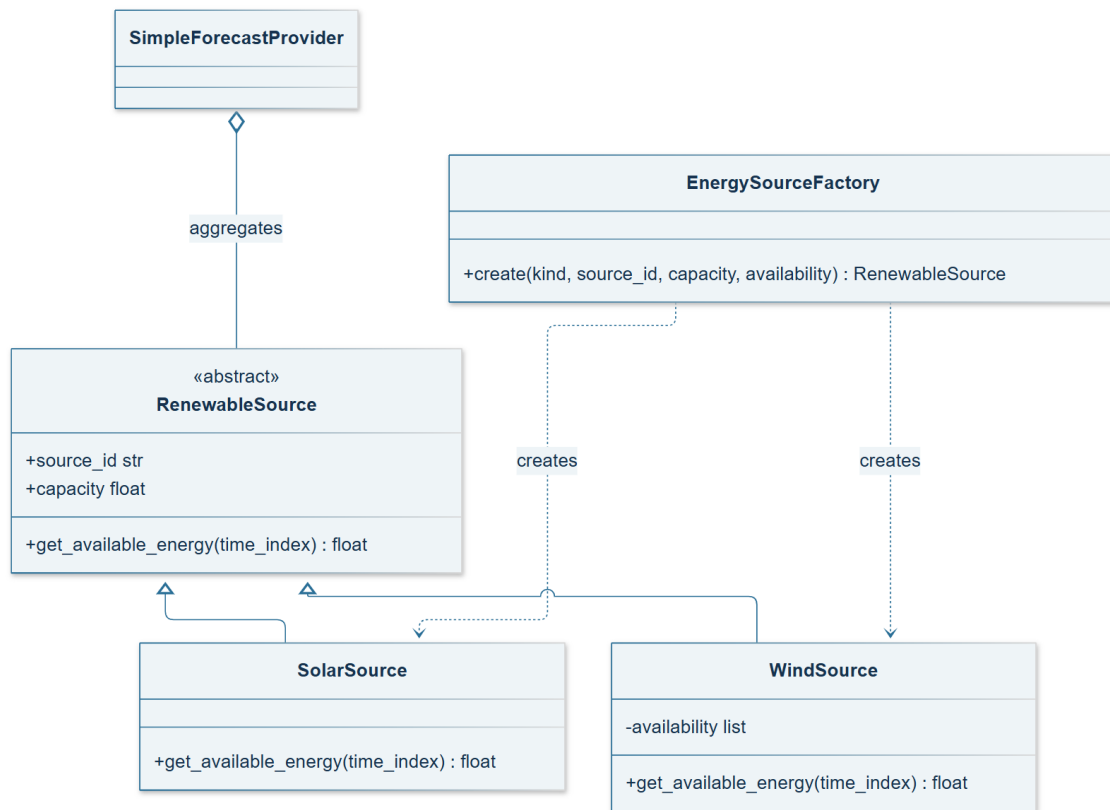


Figure 5: Class diagram for renewable-source integration.

Curl

```
curl -X 'POST' \
  'http://localhost:8000/energy-sources' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "kind": "wind",
    "source_id": "wind-turbine-1",
    "capacity": 4.0,
    "availability": [0.5, 0.8, 1.0, 0.6]
  }'
```

Request URL

```
http://localhost:8000/energy-sources
```

Server response

Code	Details
200	Response body <pre>{ "source_id": "wind-turbine-1", "kind": "wind" }</pre>

Curl

```
curl -X 'GET' \
  'http://localhost:8000/energy-sources' \
  -H 'accept: */*'
```

Request URL

```
http://localhost:8000/energy-sources
```

Server response

Code	Details
200	Response body <pre>[{ "source_id": "roof-solar-1", "kind": "SolarSource", "capacity": 8, "last_production": 0 }, { "source_id": "wind-turbine-1", "kind": "WindSource", "capacity": 4, "last_production": 2 }]</pre>

Figure 6: Swagger UI evidence for adding and retrieving renewable-energy sources.

Refactoring: Smell: adding alert rules directly to metric advancement would couple the application service to one reaction. Technique: Introduce Observer and event publication. Behavior preservation: completed metrics are still recorded before the event is published.

Before/after metrics: Pattern types remain 6. Pattern instances: 9 to 10. Full-suite result: 13 tests passed in 0.53 seconds; the MILP regression smoke test completed. A separate comparable source-LOC measurement was not captured.

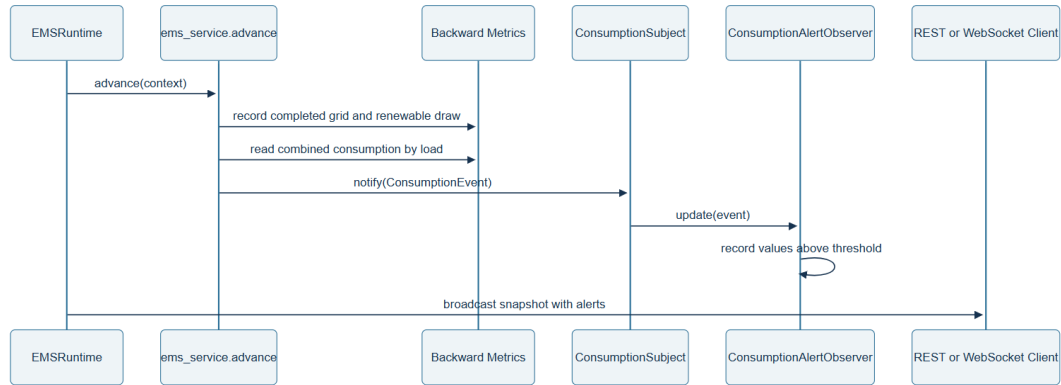


Figure 7: Sequence diagram for consumption-alert publication and notification.

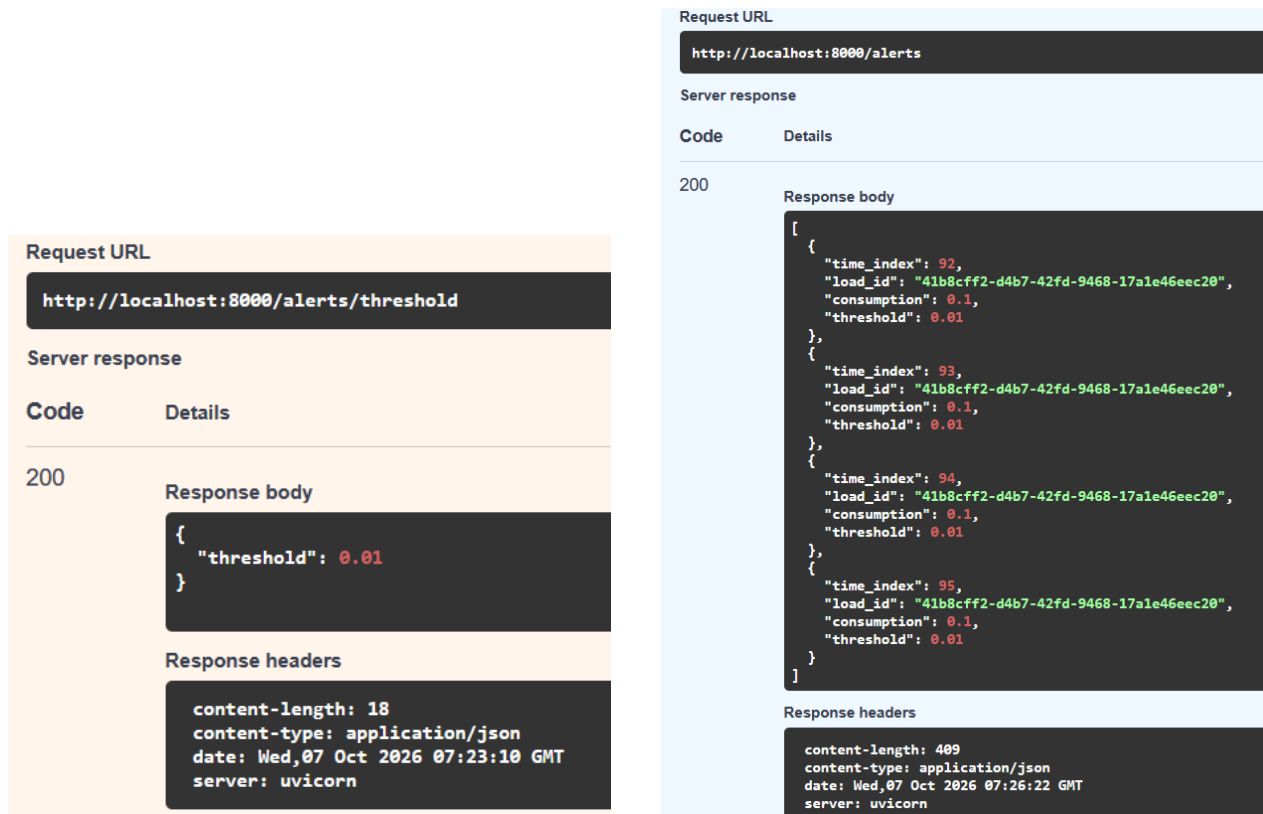


Figure 8: Swagger UI evidence for configuring and retrieving consumption alerts.

```

PS D:\B13\Part2\code\energy-management-system-updated> .\venv-win\Scripts\python.exe -m pytest -q test
ill be removed in PuLP 4.0. Install CBC with `pip install pulp[cbc]` and use COIN_CMD instead.
v4_deprecation(PULP_CBC_CMD_DEPRECATION_MSG, stacklevel=stacklevel)

.venv-win\Lib\site-packages\fastapi\testclient.py:1
D:\B13\Part2\code\energy-management-system-updated\.venv-win\Lib\site-packages\fastapi\testclient.py:1
lette.testclient` is deprecated; install `httpx2` instead.
from starlette.testclient import TestClient as TestClient # noqa

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
13 passed, 2 warnings in 0.64s

```

Figure 9: Terminal output showing the complete 13-test suite passing.

3.4 Updated pattern counts

Snapshot	Pattern types	Instances	Main effect
Original code	5	6	Baseline
After CH01	5	7	New tariff Strategy
After CH02	6	9	Renewable Strategy and Factory
After CH03	6	10	New alert Observer

Interpretation: The changes add variation at existing boundaries instead of replacing the optimization core. Strategy keeps pricing and source behavior interchangeable; Factory centralizes renewable-source construction; Observer separates consumption updates from alert rules. Trade-offs include more domain types, configuration validation, in-memory alert retention, and deterministic simulation models rather than physical telemetry. More LOC is not treated as proof of improved design.

4 Part 3 Cross-language comparison

Selected instances: Dynamic Tariff Selection (Strategy) and Consumption Threshold Alerts (Observer). They are significant because both were introduced during Part 2, have clearly identifiable participants, and can be reduced to small problems with deterministic shared cases.

Common specifications: Strategy cases cover flat prices, time-of-use peak boundaries, repeating dynamic series, context delegation, and invalid inputs. Observer cases cover below, equal, and above-threshold values, multiple observers, unsubscribe behavior, and event data. Shared fixtures are `Part3/test_cases/dynamic_tariff.csv` and `Part3/test_cases/consumption_alert.csv`.

Implementations: Each problem has Java, Python, JavaScript, and C++ production snippets with separate test runners under `Part3/code`. Java uses `javac/java`; Python uses `python`; JavaScript uses `Node.js`; C++ uses a C++17 compiler. Exact commands are recorded in `data/cross_language.csv` and `Part3/README.md`.

Actual results: Python and JavaScript output was captured in the evidence workspace. Java and C++ passes were confirmed by the team, but raw terminal output and toolchain versions were not supplied. No missing timing was estimated. Logs are under `Part3/evidence/test_results`.

Pattern	Language	LOC	Max CC	Test result	Median ms
Strategy	Java	72	5	17/17 user-confirmed	–
Strategy	Python	51	3	17/17 tool-captured	20.552
Strategy	JavaScript	71	5	17/17 tool-captured	41.596
Strategy	C++	82	5	17/17 user-confirmed	–
Observer	Java	86	3	10/10 user-confirmed	–
Observer	Python	43	3	10/10 tool-captured	45.165
Observer	JavaScript	63	3	10/10 tool-captured	42.924
Observer	C++	67	3	10/10 user-confirmed	–

Static and dynamic comparison: LOC excludes blank lines, comments, and tests. Maximum cyclomatic complexity follows the decision-counting rule documented in `Part3/evidence/metrics.md`. Timings include process startup, test execution, and output; one warmup and five measured runs were used. Raw values and medians are in `Part3/evidence/execution_times`. Python 3.12.14 and Node.js 24.21.0 were measured. Java and C++ timing fields remain blank because the required runs were unavailable.

Comparison: Python uses abstract base classes and concise dynamic dispatch, producing the smallest snippets. Java uses nominal interfaces and explicit classes. JavaScript expresses the roles compactly but validates collaborators and numeric input at runtime. C++ uses abstract classes, virtual dispatch, and explicit ownership choices. Across languages, Strategy preserves a context plus interchangeable pricing policies; Observer preserves subject subscription, event publication, and threshold observers.

Technology discussion: The language mechanisms differ, but the pattern intent remains stable. Static type systems in Java and C++ make participant contracts explicit at compile time. Python’s abstract base classes document the contract with less ceremony. JavaScript relies more heavily on runtime object behavior. The measured process times are reproducibility evidence for this environment and must not be interpreted as isolated algorithm benchmarks.

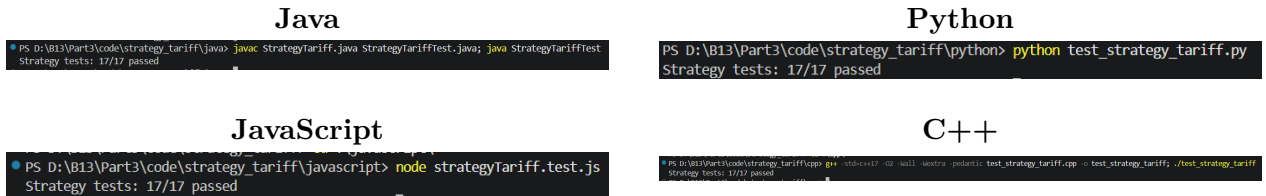


Figure 10: Terminal test results for the Dynamic Tariff Strategy implementations.



Figure 11: Terminal test results for the Consumption Alert Observer implementations.

5 LLM records and human verification

Run and model: Part 1: model and date were not recorded in the supplied transcript. Part 2: GPT-6, 2026-10-07. Part 3: GPT-6, 2026-10-07. Access-tier details were not recorded.

Inputs and outputs: Part 1 records: `Part1/llm/part1_pattern_analysis`. Part 2 records: `Part2/llm/software_evolution`. Part 3 records: `Part3/llm/cross_language_implementation`. Each directory contains `prompt.txt`, `input.txt`, `output.txt`, and `verification.md`. The consolidated index is `data/llm.csv`.

Claims and evaluation: Part 1 was partially correct: capability classifications and candidates were checked against source, and exact roles and locations were corrected in `data/patterns.csv`. Part 2 was correct after human verification through 13 tests and a MILP smoke test. Part 3 was partial: semantics and Python/JavaScript runs were confirmed; Java/C++ passes are team-confirmed, while raw timings and toolchain versions remain unavailable. Reviewers: Team B13.

Generated code: Part 2 generated changes were validated through the full application test suite and MILP smoke test. Part 3 generated snippets were checked against shared cases; Python and JavaScript were executed in the evidence workspace, and Java/C++ execution was confirmed by the team.

6 Findings, contributions and references

Findings: The baseline already contains a coherent scheduling core and several architectural collaborations. Part 2 adds dynamic behavior at existing seams: tariff and renewable algorithms become Strategies, renewable construction uses a Factory, and alerts use Observer. Part 3 shows that the participant relationships survive translation across four languages, while source size, type declarations, ownership, and runtime validation differ. Limitations include simulated data, no physical smart-meter adapter, no user/authentication domain, in-memory alerts, no email/SMS delivery, and missing Java/C++ raw timing evidence.

Contributions

Roll number	Name and batch	Actual contribution / evidence link
AM.SC.U4CSE23306	Anand Narayan, CSE	Analysed the original application architecture and identified the baseline design patterns; prepared the Part 1 architecture and pattern evidence; implemented and verified the Java Strategy and Observer examples for Part 3; jointly reviewed the report and datasets. Evidence: <code>Part1/evidence/architecture/</code> , <code>Part1/evidence/patterns/</code> , and <code>Part3/code/*/java/</code> .
AM.SC.U4CSE23346	S H Saimadhav, CSE	Designed and implemented the dynamic-tariff evolution using the Strategy pattern; prepared the Change 1 requirement, source mapping, before/after analysis, diff, and test evidence; implemented and verified the Python Strategy and Observer examples for Part 3; jointly validated the final report and CSV records. Evidence: <code>Part2/evidence/changes/Change1/</code> , <code>Part2/code/ems/domain/tariff/</code> , and <code>Part3/code/*/python/</code> .
AM.SC.U4CSE23365	Vaishnav P Nair, CSE	Designed and implemented solar and wind renewable-source integration using Strategy and Factory; prepared the Change 2 requirement, source mapping, before/after analysis, diff, and test evidence; implemented and verified the JavaScript Strategy and Observer examples for Part 3; jointly checked UML diagrams and repository organization. Evidence: <code>Part2/evidence/changes/Change2/</code> , <code>Part2/code/ems/domain/energy_sources/</code> , and <code>Part3/code/*/javascript/</code> .
AM.SC.U4CSE23369	Vishnu M, CSE	Designed and implemented consumption alerts using the Observer pattern; prepared the Change 3 requirement, source mapping, before/after analysis, diff, and test evidence; implemented and verified the C++ Strategy and Observer examples for Part 3; jointly performed regression testing and reviewed the final metrics and documentation. Evidence: <code>Part2/evidence/changes/Change3/</code> , <code>Part2/code/ems/domain/alerts/</code> , and <code>Part3/code/*/cpp/</code> .

References

References

- [1] FastAPI documentation, <https://fastapi.tiangolo.com/>.
- [2] Python documentation, *abc — Abstract Base Classes*, <https://docs.python.org/3/library/abc.html>.

- [3] Oracle Java documentation, *Interfaces*, <https://docs.oracle.com/javase/tutorial/java/IandI/createinterface.html>.
- [4] MDN Web Docs, *JavaScript classes*, <https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Classes>.
- [5] cppreference, *Virtual functions*, <https://en.cppreference.com/w/cpp/language/virtual.html>.
- [6] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1994.
- [7] Supplied application archives and evidence package, Team B13, inspected 2026-10-07.

7 Team folder and datasets

B13/

```
README.md
B13_Report.tex
B13_Report.pdf
B13_Slides.pptx
data/
  patterns.csv
  pattern_counts.csv
  patterns_evolution.csv
  cross_language.csv
  llm.csv
Part1/
  code/
  evidence/
    architecture/
    patterns/
  llm/
Part2/
  code/
  evidence/
    changes/Change1/
    changes/Change2/
    changes/Change3/
    diagrams/
  llm/
Part3/
  code/
  test_cases/
  evidence/
    test_results/
    execution_times/
  llm/
```

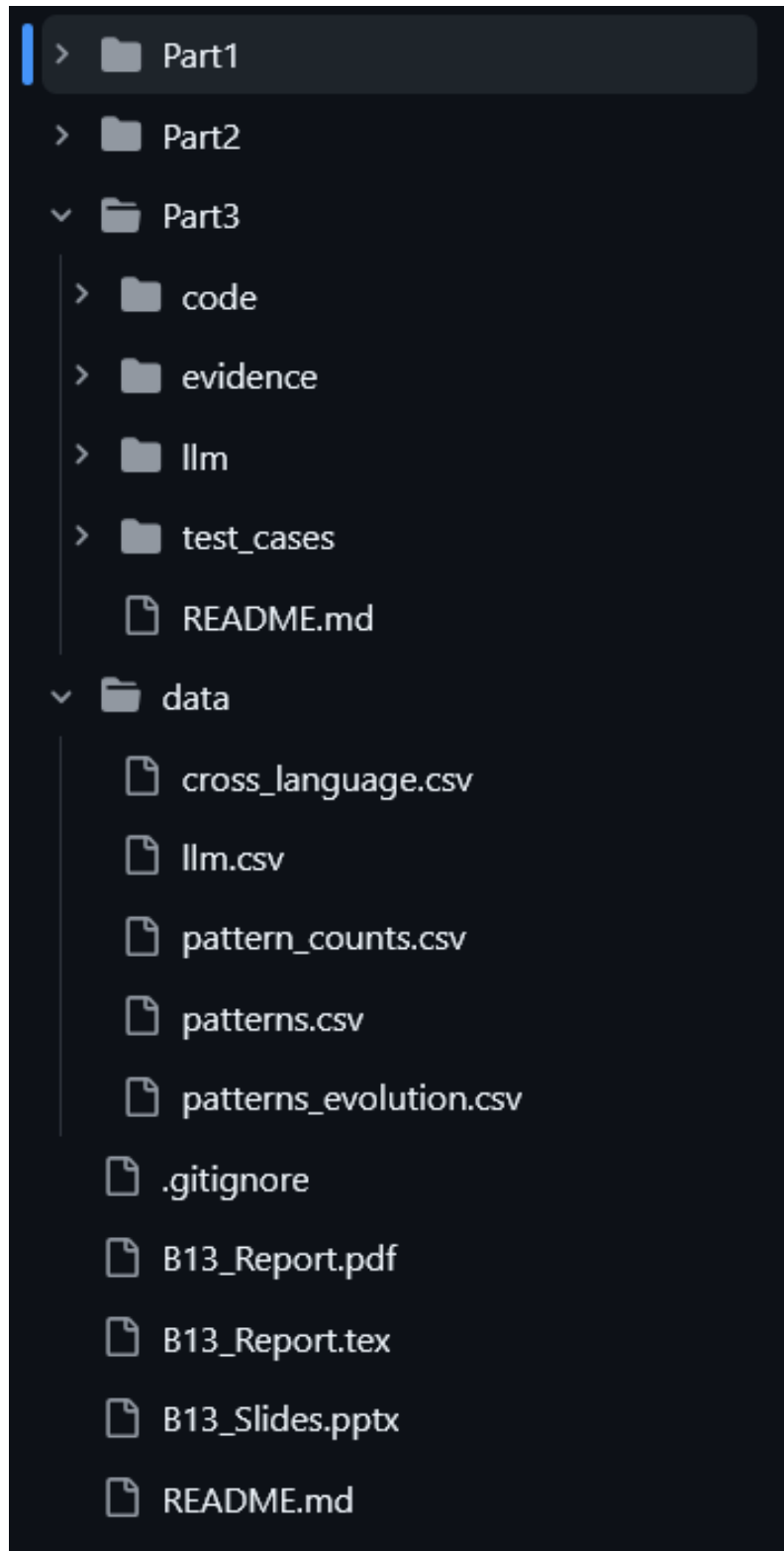


Figure 12: Complete GitHub repository structure for the B13 team folder, including the README, reports, consolidated datasets, and supporting artifacts for Parts 1, 2, and 3.